# 시계열 예측 개요

정량적 시계열 예측의 이해

TIME-SERIES

저자
Chunghun Ha

소속
Hongik University

공개
2024년 4월 13일

MODIFIED
2026년 3월 8일

# 시계열 예측

## 시계열 예측(Time-Series Forecasting)

> ***과거의 시계열 데이터***를 토대로 ***미래의 시계열 데이터***를 예측하는 것

- 주로 정량적 데이터에 대한 정량적 예측을 의미

## 시계열 예측의 중요성

- ***미래를 위한 전략 수립***
  - 미래를 위한 전략 수립의 토대이자 중요한 의사 결정을 위한 기반 자료로 활용
- ***적용 분야의 다양성***
  - 금융, 마케팅, 생산, 유통, 의료, 환경 등 다양한 분야에서 요구
- ***비즈니스 의사 결정***
  - 매출 예측, 재고 관리, 수요 예측 등 다양한 비즈니스 의사 결정에 활용

## 시계열 예측이 어려운 점

- ***예측의 불확실성***
  - 시계열 예측은 미래의 데이터를 예측하는 것이기 때문에 불확실성이 필연적으로 존재
  - 따라서 확률적 방법론을 사용하여 예측의 불확실성을 고려하고 평가 하여야 함
- ***데이터의 노이즈***
  - 시계열 데이터는 노이즈가 많이 포함되어 있기 때문에 노이즈를 제거하고 신뢰할 수 있는 데이터를 추출하는 것이 중요
- ***패턴의 다양성***

  - 시계열 데이터는 추세, 계절성, 주기 등 다양한 유형의 패턴을 가지고 있기 때문에 데이터가 충분하지 않거나, 노이즈가 많은 경우 패턴을 찾아내기 어려움
- ***다변량 시계열***
  - 다변량 시계열은 여러 변수들 간의 상관 관계가 복잡하게 얽혀 있고, 이들 간에 시차 또한 존재하므로 이를 고려하여 예측을 해야 함
  - 최근 데이터 수집 기술의 발전으로 다변량 시계열 데이터가 많아져 다변량 시계열 예측의 중요성이 증가

# 시계열 예측 프로세스

1. ***데이터 수집 및 전처리***
   - 시계열 데이터를 수집하고, 필요한 경우 누락된 데이터나 이상치를 처리하고 정규화하여 데이터를 준비
2. ***시각화와 탐색적 데이터 분석 (EDA)***
   - 데이터를 그래프나 통계적 분석을 통해 시각화하고, 패턴이나 트렌드, 계절성 등을 탐색
   - 이 단계에서 데이터의 특성을 이해하는 것이 중요
3. ***예측 모형 선택***
   - 시계열 데이터의 특성과 예측 목표와 수준에 따라 적합한 예측 모형을 선택
   - ARIMA(AutoRegressive Integrated Moving Average), SARIMA(Seasonal ARIMA), Prophet, LSTM(Long Short-Term Memory), GRU(Gated Recurrent Unit) 등 다양한 모델 중 적합한 모델을 선택
4. ***예측 모형 학습***
   - 선택한 모형에 데이터를 입력하고, 모형을 학습시킴
5. ***모형 평가***
   - 학습된 모형을 평가 데이터에 적용하여 예측 성능을 평가
   - 평가 지표로는 평균 제곱근 오차(RMSE, Root Mean Squared Error), 평균 절대 오차(MAE, Mean Absolute Error), R-squared 등이 있음
6. ***예측 적용***
   - 학습된 모형을 사용하여 미래 값을 예측
   - 이 때, 예측 구간과 신뢰 구간을 고려하여 결과를 해석하는 것이 중요

# 시계열 예측 모형의 역사

📑 **참고**: A Brief History of Time Series Models[1]

- 1950s — Exponential Smoothing: 전통적인 **지수평활법**
- 1970s — **ARIMA**(AutoRegressive Integrated Moving Average): 자기회귀모형+누적차분+이동평균모형
- 1980 — **VAR**(Vector Autoregression): 다변량 시계열 모형
- 1980s — **GARCH**(Generalized Autoregressive Conditional Heteroskedasticity): 변동성(분산)을 예측하는 모형
- 2000 — **Theta** model: 지수평활과 국소 곡률을 결합한 모형
- 2011 — **TBATS**(Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend and Seasonal components): 지수평활과 ARMA 오차항을 결합한 모형
- 2014 — **XGBoost** (Extreme Gradient Boosting): Gradient Boosting의 확장
- 2016 — **LightGBM**: Microsoft, 속도 개선 로직과 정규화 방법을 보유하고 있는 Gradient Boosting의 확장
- 2017 — CatBoost: Yandex, 속도 개선 로직과 정규화 방법을 보유하고 있는 Gradient Boosting의 확장
- 2017 — **Prophet**: Facebook에서 개발한 시계열 예측 모형
- 2017 — DeepAR: Amazon, RNN 계열의 딥러닝 모형
- 2017 — MQRNN(Multi-Horizon Quantile Recurrent Forecaster): Amazon, RNN 계열의 딥러닝 모형

- 2018 — DSSM(Deep State Space Model): Amazon, SSM(확률과정을 이용하는 모형)과 RNN을 결합한 모형
- 2018 — TCN(Temporal Convolution Network): CMU, CNN 계열의 딥러닝 모형
- 2018 — T-GCN(Temporal Graph Convolutional Network): Central South University(China), CNN 계열의 딥러닝 모형
- 2018 — ESRNN(Exponential Smoothing Recurrent Neural Network): Uber, RNN 계열의 딥러닝 모형
- 2019 — N-BEATS(Neural Basis Expansion Analysis for Interpretable Time Sereis Forecasting): ElementAI, RNN 계열의 딥러닝 모형
- 2019 — AR-Net(A simple auto-regressive Neural Network for time-series): Facebook, RNN 계열의 딥러닝 모형
- 2020 — NeuralProphet: Facebook, RNN 계열의 딥러닝 모형
- 2020 — TFT(Temporal Fusion Transformer): Google, Transformer 계열의 딥러닝 모형
- 2021 — ThymeBoost: Microsoft, Gradient Boosting의 확장
- 2021 — Greykite/Silverkite: LinkedIn, Prophet의 확장
- 2021 — Orbit(Object oRiented BayesIan Time series): Uber, 베이지안 시계열 모델
- 2022 — N-HiTS(Neural Hierarchical Interpolation for Time Series Forecasting): CMU/Unity Technologies/Nixtla, CNN 계열의 딥러닝 모형
- 2023 — PatchTST(channel-independent patch time series Transformer), Princeton/IBM, 다변량 시계열을 위한 Transformer 계열의 딥러닝 모형

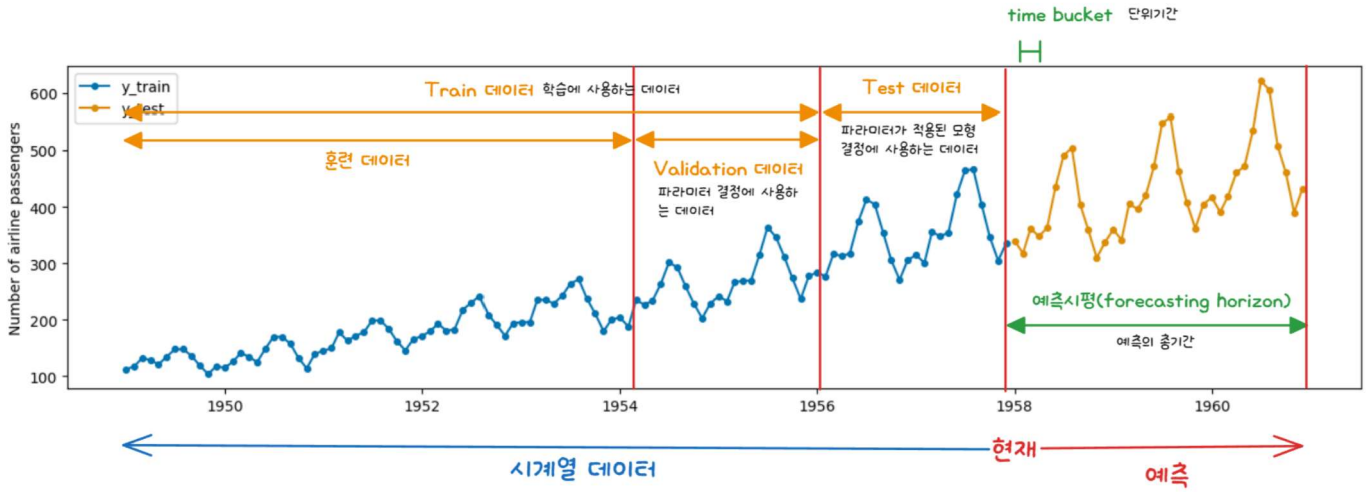
> ⓘ 통계적 모형과 ML모형
>
> - **단변량 예측은 통계적 모형**이 잘 동작하고 **다변량 예측은 ML 계열**이 유리
> - 주류 모형은 통계적 모형에서 ML 계열로 이동

# 시계열 예측의 평가

## 예측의 평가방법

- 예측의 평가는 ***오차(예측값과 실제값의 차이)*** 를 측정하여 수행
- 예측값은 미래에 대해서만 존재 ➔ 데이터를 추정(학습)데이터와 평가데이터로 분리
  - 통상적으로 8:2 비율을 사용하나 예측시평의 크기를 고려하여 결정
  - 예측시평이 짧은 단기예측의 경우에는 Rolling Horizon(한 기간씩 이동하며 평가)으로 평가

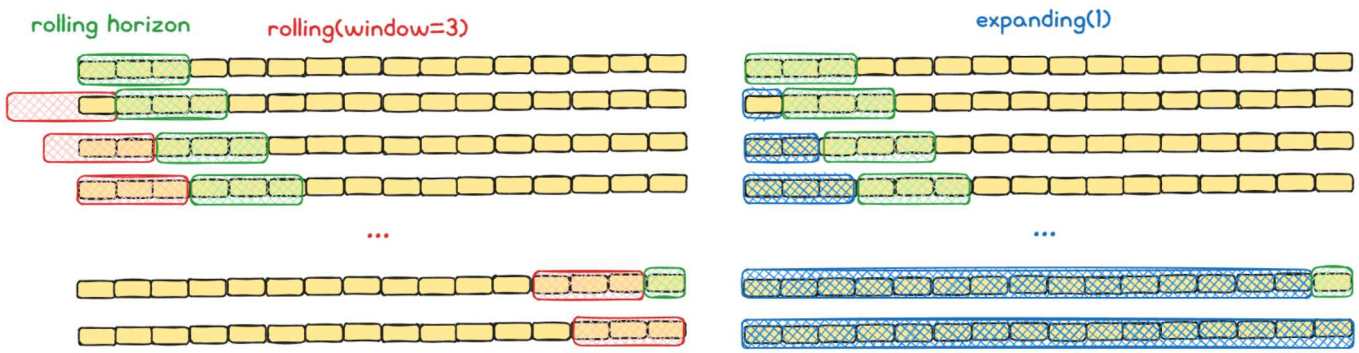
📊 그림: 시계열 예측모형의 평가를 위한 분리



# 예측의 평가구간

- 시계열 예측은 주기적으로(반복적으로) 시행
  - 데이터의 크기를 동일하게 사용하면, Rolling을 사용
  - 누적된 모든 데이터를 사용하면, Expanding을 사용
- 예측의 평가구간은 예측값과 실제값을 비교하는 구간
  - 시평기간의 예측성능이 중요하므로, 예측의 평가구간은 시평을 사용
  - Validation 데이터나 Test 데이터의 크기가 시평보다 크면, Rolling Horizon을 사용
  - Validation 데이터나 Test 데이터의 크기가 시평보다 작으면, 전체 구간에 대하여 평가

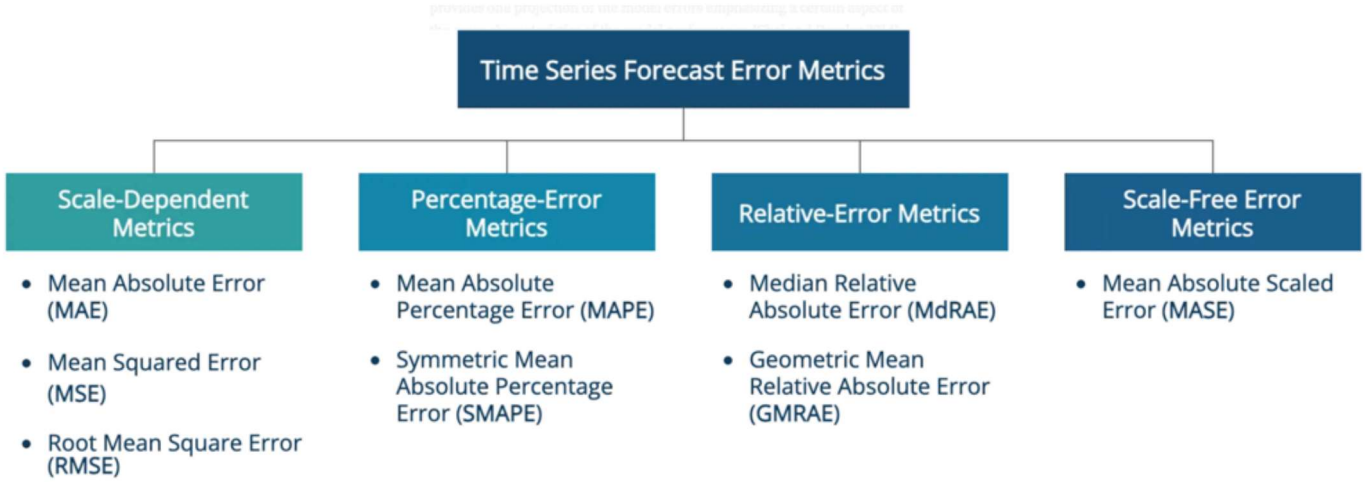
📊 그림: 예측의 평가구간




# 성능평가 지표

- 오차($e_t = y_t - f_t$)에 대한 다양한 지표로 평가
  - $y_t$: 실제값, $f_t$: 예측값
- 다양한 평가지표가 있으므로 수요예측의 목적에 맞게 다양한 조합을 적용하여 평가

📊 그림: 예측 평가지표[2]




자주 사용하는 수요예측 평가지표

| 지표 | Fullname | 수식 | 특징 |
|---|---|---|---|
| MAD(MAE) | mean absolute deviation | $\frac{1}{n}\sum_{t=1}^{n}\lvert e_t\rvert$ | MAD=0 이면 예측이 정확함. 아니면 과대 혹은 과소 예측을 의미 |
| MSE | mean squared error | $\frac{1}{n}\sum_{t=1}^{n}{e_t}^2$ | 오차 제곱의 평균, 통계학의 분산과 유사 |

| 지표 | Fullname | 수식 | 특징 |
|---|---|---|---|
| RMSE | root mean squared error | $\sqrt{\frac{1}{n}\sum_{t=1}^{n} {e_t}^2}$ | MSE의 제곱근, 예측치와 단위가 동일 |
| MAPE | mean absolute percentage error | $\frac{100}{n}\sum_{t=1}^{n}\lvert\frac{e_t}{y_t}\rvert$ | 절대 백분률 오차의 평균, 서로 다른 척도간 오차를 측정 가능, 0~100% |
| SMAPE | symmetric mean absolute percentage error | $\frac{100}{n}\sum_{t=1}^{n}\frac{\lvert e_t\rvert}{(\lvert y_t\rvert+\lvert \hat{y}_t\rvert)/2}$ | 대칭적 절대 백분률 오차의 평균, 서로 다른 척도간 오차를 측정 가능, 0~200% |
| MdRAE | median relative absolute error | $median(r_1,...,r_t), r_t=\lvert\frac{e_t}{y_t-y_{t-m}}\rvert$ | 벤치마크 예측값($m$은 계절주기, 비계절은 $m=1$)에 대한 상대절대오차의 중앙값, $y_t$가 0 근방에서도 잘 동작 |
| GMRAE | geometric mean relative absolute error | $\sqrt[n]{\Pi_{t-1}^{n} r_t}, r_t=\lvert\frac{e_t}{y_t-y_{t-m}}\rvert$ | 벤치마크 예측값($m$은 계절주기, 비계절은 $m=1$)에 대한 상대절대오차의 기하평균, $y_t$가 0 근방에서도 잘 동작 |
| MASE | mean absolute scaled error | $\frac{1}{n}\sum_{t=1}^{n}\frac{\lvert e_t\rvert}{\frac{1}{n-m}\sum_{t=2}^{n}\lvert y_t-y_{t-m}\rvert}$ | 벤치마크 예측값($m$은 계절주기, 비계절은 $m=1$)에 대한 평균, $y_t$가 0 근방에서도 잘 동작 |
| RSFE | running sum of forecast error | $\sum_{t=1}^{n} e_t$ | 예측의 편향성 지표 |
| TS | tracking signal | $\frac{RSFE}{MAD}$ | 예측의 편향성이 허용범위 내에 있는지 검토할 때 사용 |

### 성능지표 선택 가이드

#### 지표별 특성 비교

| 지표 | 스케일 의존 | 이상치 민감 | 0 근방 안정 | 주요 용도 |
|---|---|---|---|---|
| MAE | ✅ | 보통 | ✅ | 일반적인 오차 크기 평가 |
| MSE/RMSE | ✅ | 높음 | ✅ | 큰 오차에 패널티 부여 시 |
| MAPE | ❌ (%) | 보통 | ❌ ($y_t \approx 0$이면 발산) | 서로 다른 척도 간 비교 |
| SMAPE | ❌ (%) | 보통 | 보통 | MAPE의 비대칭 문제 보완 |
| MdRAE/GMRAE | ❌ (비율) | 낮음 | ✅ | 벤치마크 대비 상대 성능 평가 |
| MASE | ❌ (비율) | 보통 | ✅ | $y_t \approx 0$인 경우에도 안정적 |
| RSFE/TS | ✅ | – | – | 편향성(과대/과소 예측) 진단 |

#### 선택 기준 요약

- 단일 시계열 평가 → MAE 또는 RMSE (직관적이고 해석 용이)
- 서로 다른 척도 비교 → MAPE (단, $y_t \approx 0$이면 MASE 사용)
- 이상치에 강건한 평가 → MdRAE (중앙값 기반)
- 편향 진단 → TS (|TS| > 4이면 편향 의심)

Rolling vs Expanding 비교

| 구분 | Rolling | Expanding |
|---|---|---|
| 학습 데이터 | 고정 크기 윈도우 (최신 $n$개) | 누적 (처음부터 현재까지) |
| 장점 | 최신 패턴에 민감, 개념 드리프트 대응 | 데이터 활용 극대화, 안정적 |
| 단점 | 과거 장기 패턴 무시 가능 | 오래된 데이터가 잡음이 될 수 있음 |
| 적합 상황 | 패턴이 자주 변하는 시계열 | 장기적으로 안정된 시계열 |

# sktime 라이브러리의 시계열 예측 평가지표

## 기본 사용법

- `sktime.performance_metrics.forecasting` 모듈에서 예측에 대한 평가지표를 제공
  - 클래스: CamelCase로 작성되어 있음. 객체를 생성하여 사용하여야 함
    - 예) `MeanAbsolutePercentageError`, `MeanAbsoluteError`, `MeanSquaredError`, ...
  - 함수: snake_case로 작성되어 있음. 함수를 호출하여 사용하여야 함
    - 예) `mean_absolute_percentage_error`, `mean_absolute_error`, `mean_squared_error`, ...

```python
import pandas as pd
import koreanize_matplotlib
import sktime.performance_metrics.forecasting as sktm
from sktime.performance_metrics.forecasting import mean_absolute_percentage_error as MAPE

ts = pd.Series([10, 15, 9, 12, 8, 11, 14, 13, 10, 12])
ts_pred = pd.Series([10, 12, 8, 11, 14, 13, 10, 12, 9, 15])

# 클래스 사용
mape = sktm.MeanAbsolutePercentageError()
print(f'{mape(ts, ts_pred) = }')

# 함수로 사용
print(f'{sktm.mean_absolute_percentage_error(ts, ts_pred) = }')

# 함수로 사용 (alias)
print(f'{MAPE(ts, ts_pred) = }')
```

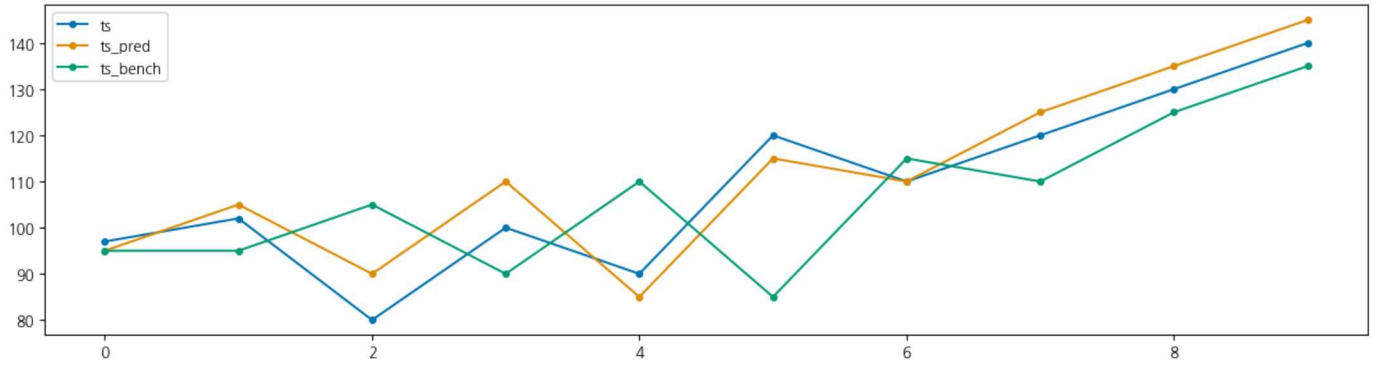
```
mape(ts, ts_pred) = np.float64(0.20388999888999892)
sktm.mean_absolute_percentage_error(ts, ts_pred) = np.float64(0.20388999888999892)
MAPE(ts, ts_pred) = np.float64(0.20388999888999892)
```

## sktime 라이브러리에서 제공하는 평가지표

⏩ 🌏 API Reference > Performance metrics

```python
import pandas as pd
from sktime.utils.plotting import plot_series
from sktime.performance_metrics.forecasting import mean_absolute_error as MAE
from sktime.performance_metrics.forecasting import median_absolute_error as MdAE
from sktime.performance_metrics.forecasting import mean_squared_error as MSE
from sktime.performance_metrics.forecasting import median_squared_error as MdSE
from sktime.performance_metrics.forecasting import mean_absolute_percentage_error as MAPE
from sktime.performance_metrics.forecasting import median_relative_absolute_error as MdRAE
from sktime.performance_metrics.forecasting import geometric_mean_relative_absolute_error as GMRAE
from sktime.performance_metrics.forecasting import mean_absolute_scaled_error as MASE

ts = pd.Series([97, 102, 80, 100, 90, 120, 110, 120, 130, 140])
ts_pred = pd.Series([95, 105, 90, 110, 85, 115, 110, 125, 135, 145])
# ts_pred를 한 시점씩 이동하고 결측치를 마지막 값으로 대체
ts_bench = ts_pred.shift(1).bfill()

# 시계열을 시각화
plot_series(ts, ts_pred, ts_bench, labels=["ts", "ts_pred", "ts_bench"])

print(f'{MAE(ts, ts_pred) = :.4f}')
print(f'{MdAE(ts, ts_pred) = :.4f}')
print(f'{MSE(ts, ts_pred) = :.4f}')
print(f'{MdSE(ts, ts_pred) = :.4f}')
print(f'{MAPE(ts, ts_pred) = :.4f}')
print(f'{MAPE(ts, ts_pred, symmetric=True) = :.4f}') # SMAPE
print(f'{MdRAE(ts, ts_pred, y_pred_benchmark=ts_bench) = :.4f}')
print(f'{GMRAE(ts, ts_pred, y_pred_benchmark=ts_bench) = :.4f}')
print(f'{MASE(ts, ts_pred, y_train=ts_bench) = :.4f}')
print(f'RSFE = {(ts - ts_pred).sum():.4f}')
print(f'TS = {(ts - ts_pred).sum()/MAE(ts, ts_pred):.4f}')
```

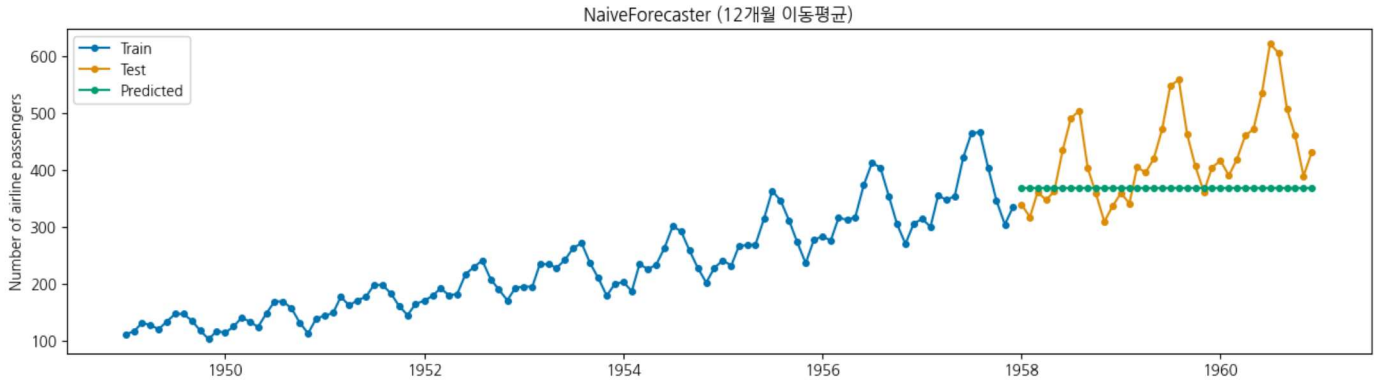
```
MAE(ts, ts_pred) = 5.0000
MdAE(ts, ts_pred) = 5.0000
MSE(ts, ts_pred) = 33.8000
MdSE(ts, ts_pred) = 25.0000
MAPE(ts, ts_pred) = 0.0488
MAPE(ts, ts_pred, symmetric=True) = 0.0476
MdRAE(ts, ts_pred, y_pred_benchmark=ts_bench) = 0.4643
GMRAE(ts, ts_pred, y_pred_benchmark=ts_bench) = 0.0152
MASE(ts, ts_pred, y_train=ts_bench) = 0.3462
RSFE = -26.0000
TS = -5.2000
```




# 간단한 시계열 예측 예제

- sktime의 `NaiveForecaster`를 사용하여 시계열 예측의 전체 프로세스를 실습
  - NaiveForecaster: 과거 데이터를 그대로 반복하거나 이동평균으로 예측하는 단순 기준모형(baseline model)
  - 시계열 예측의 기본 흐름: 데이터 분할 → 모형 학습 → 예측 → 평가

```python
import pandas as pd
import matplotlib.pyplot as plt
from sktime.datasets import load_airline
from sktime.split import temporal_train_test_split
from sktime.forecasting.naive import NaiveForecaster
from sktime.performance_metrics.forecasting import mean_absolute_percentage_error as MAPE
from sktime.utils.plotting import plot_series

# 항공 승객 데이터 로드
y = load_airline()

# 학습/평가 데이터 분할 (80:20)
y_train, y_test = temporal_train_test_split(y, test_size=36)

# NaiveForecaster: 최근 12개월의 평균으로 예측 (이동평균 기준모형)
forecaster = NaiveForecaster(strategy="mean", window_length=12)
forecaster.fit(y_train)

# 평가 데이터 기간만큼 예측
fh = list(range(1, len(y_test) + 1))
y_pred = forecaster.predict(fh=fh)

# 결과 시각화
plot_series(y_train, y_test, y_pred, labels=["Train", "Test", "Predicted"])
plt.title("NaiveForecaster (12개월 이동평균)")
plt.show()

# 성능 평가
print(f"MAPE = {MAPE(y_test, y_pred):.4f}")
```




```
MAPE = 0.1552
```

> 시계열 예측의 기본 흐름
>
> 1. **데이터 분할**: `temporal_train_test_split()` 으로 시간 순서를 유지하며 분할
> 2. **모형 학습**: `forecaster.fit(y_train)` 으로 과거 데이터 학습
> 3. **예측**: `forecaster.predict(fh)` 로 미래 시점 예측
> 4. **평가**: `MAPE` 등의 지표로 예측 성능 측정
>
> - NaiveForecaster는 **기준모형(baseline)**으로, 더 복잡한 모형의 성능을 비교하는 기준으로 활용

---

## 각주

1. https://medium.com/@ycwong.joe/a-brief-history-of-time-series-models-38455c2cd78d ↩
2. https://towardsdatascience.com/time-series-forecast-error-metrics-you-should-know-cc88b8c67f27 ↩

Copyright 2026, Chunghun Ha, Hongik University